

Argo CD tutorial (running on Ubuntu with Java 25, Spring Boot, Kubernetes with Kind, and GitLab)

→ Disclaimer ←

This tutorial is provided as is, without any warranty from the [\[author\]](#). It's intended for educational purposes only. You can **download the latest version of this tutorial FOR FREE (in zip format)** and follow the instructions in the [README.{html,pdf}](#) files (this one you are reading). To do that, use the following link:

<https://paulojeronimo.com/tmp/argo-cd-tutorial.zip>

You are reading the PDF version.
But you can also read [this tutorial in HTML format](#).

Author: [Paulo Jerônimo](#)

Last update: 2025-10-17 03:17:37 -0300

Last Git commit: 8c7a21e

1. Introduction	3
2. Prerequisites	4
2.1. Ubuntu installation	4
2.2. Docker installation	5
2.3. Kubectl installation	5
2.4. Kind installation	6
2.5. SDKMAN installation	7
2.6. Java 25 installation	7
2.7. Argo CD installation	8
3. Download and "install" this tutorial	10
4. Spring Boot initial project creation	11
5. Application development with Java 25 features	13
5.1. Gradle Wrapper update	13
5.2. Creating the EchoController class	14
5.3. Updating the EchoController class adding a Java 25 feature	16
6. Labs	17
6.1. Test a Kubernetes cluster with a load balancer	17

6.2. Getting started with Argo CD	23
7. References	26

1. Introduction

By creating this tutorial, I intended to make some commands I use on Linux ([Ubuntu](#)) to install and configure [Argo CD](#) on a [Kubernetes](#) cluster managed by [Kind](#) visible.

For illustrative purposes only (not the main purpose of this tutorial), I'm using [Argo CD](#) to manage the continuous deployment of a [Java 25](#) application whose repository is in [GitLab](#).

In this tutorial, I don't use pre-built Bash scripts to prepare the environment (as I like to do in my daily life). Therefore, the focus of this tutorial is more EDUCATIONAL (aimed at Windows users) than OPERATIONAL. In the future, I plan to create a version that follows this latter approach, using many more Bash functions and scripts.

2. Prerequisites

2.1. Ubuntu installation

1. Please choose a method to install Ubuntu and proceed with it.
 - a. I am assuming that you'll be running in a fresh Ubuntu installation.
 - i. In my case, I'll be running a VM running through [QEMU/KVM](#).
 - b. If you're using Windows, my recommendation is: install Ubuntu in a [WSL](#) environment.
2. This is latest version of Ubuntu where the following commands were tested:

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:       24.04
Codename:      noble
```

2.1.1. Install some useful packages (used by next steps)

1. Open a new Bash terminal and run these commands:

```
$ sudo apt update && sudo apt install -y \
tree \
curl \
vim \
bat \
git \
xsel
```

2.1.2. Do some simple configurations

1. Configure your prompt to the most simple format:

```
$ export PS1='$ '
$ echo "!!" >> ~/.bashrc
```

2. Configure `sudo` to not ask confirmations:

```
$ echo "$(whoami) ALL=(ALL) NOPASSWD:ALL" | sudo tee /etc/sudoers.d/$(whoami)
>/dev/null && sudo chmod 440 /etc/sudoers.d/$(whoami)
```

3. Configure your default editor to `vim`:

```
$ export EDITOR=vim
$ echo "!!" >> ~/.bashrc
```

4. Add and load some useful aliases (and functions):

```
$ cat >> ~/.bash_aliases<<EOF
alias cat=batcat
alias k=kubectl
pbcopy() { xsel --clipboard --input; }
pbpaste() { xsel --clipboard --output; }
EOF
$ source ~/.bash_aliases
```

5. Configure Git:

```
$ git config --global --edit
$ git config --global diff.tool vimdiff
$ git config --global --add difftool.prompt false
$ git config --global alias.alias "! git config --get-regexp ^alias\\. | sed -e
s/^alias\\.\\.// -e s/\\ /\\ =\\ /"
$ git config --global alias.log-adog "log --all --decorate --oneline --graph"
```

2.2. Docker installation

1. Install and test `docker`:

```
$ bash <(curl -fsSL https://get.docker.com)
$ sudo docker version
$ sudo usermod -aG docker $USER
$ newgrp docker
$ docker run hello-world
```

2. Logout and login again:

```
$ loginctl terminate-user $USER
```

3. Read the [Docker references](#) to learn more.

2.3. Kubectl installation

1. Install and test `kubectl`:

```
$ curl -LO "https://dl.k8s.io/release/$(curl -L -s
```

```
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
$ sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
$ rm kubectl
$ kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
```

2. Read the [Kubectl references](#) to learn more.

2.4. Kind installation

1. Install **kind**:

```
$ curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.30.0/kind-linux-amd64
$ chmod +x ./kind
$ sudo mv ./kind /usr/local/bin/kind
```

2. Test some simple **kind** and **kubectl** (k) commands:

```
$ kind get clusters
No kind clusters found.

$ kind create cluster --name demo-cluster

$ kind get clusters
demo-cluster

$ k config get-contexts # remember: k is an alias to kubectl
CURRENT  NAME                CLUSTER             AUTHINFO             NAMESPACE
*         kind-demo-cluster   kind-demo-cluster   kind-demo-cluster

$ k get pods
No resources found in default namespace.

$ k get namespaces # same as 'k get ns'
NAME                STATUS    AGE
default             Active   62s
kube-node-lease     Active   62s
kube-public         Active   62s
kube-system         Active   63s
local-path-storage  Active   57s

$ k get pods -n kube-system
NAME                                READY  STATUS   RESTARTS
AGE
coredns-66bc5c9577-hfdk            1/1    Running   0
25s
coredns-66bc5c9577-xhkrb           1/1    Running   0
```

```

25s
etcd-demo-cluster-control-plane      1/1      Running    0
33s
kindnet-tkssm                        1/1      Running    0
26s
kube-apiserver-demo-cluster-control-plane  1/1      Running    0
33s
kube-controller-manager-demo-cluster-control-plane  1/1      Running    0
33s
kube-proxy-kfbtd                     1/1      Running    0
26s
kube-scheduler-demo-cluster-control-plane  1/1      Running    0
33s

$ kind delete cluster --name demo-cluster
Deleting cluster "demo-cluster" ...
Deleted nodes: ["demo-cluster-control-plane"]

$ k config get-contexts

```

3. At this point, if you have little knowledge in [Kubernetes](#) you may want to [Test a Kubernetes cluster with a load balancer](#) to learn a little more about it before proceeding with the steps in this tutorial.
4. Read the [Kind references](#) to learn more.

2.5. SDKMAN installation

1. Install **sdk**:

```

$ curl -s "https://get.sdkman.io" | bash
$ source ~/.sdkman/bin/sdkman-init.sh
$ sdk version

```

2. Read the [SDKMAN references](#) to learn more.

2.6. Java 25 installation

1. Install **java 25**:

```

$ sdk list java | grep 25

$ sdk install java 25-tem
....
Do you want java 25-tem to be set as default? (Y/n): Y

Setting java 25-tem as default.

```

```
$ java -version
openjdk version "25" 2025-09-16 LTS
OpenJDK Runtime Environment Temurin-25+36 (build 25+36-LTS)
OpenJDK 64-Bit Server VM Temurin-25+36 (build 25+36-LTS, mixed mode, sharing)
```

2. Read the [Java 25 references](#) to learn more.

2.7. Argo CD installation

1. Install the **argocd** Kubernetes namespace:

```
$ k create namespace argocd
$ k apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

2. Install the **argocd** CLI:

```
$ curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
$ sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
$ rm argocd-linux-amd64
```

3. Run Argo CD and expose its port 8080:

```
$ k patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
$ log=~/.argocd.log; k port-forward svc/argocd-server -n argocd 8080:443 &> $log &
$ tail -f $log # and press Ctrl+C
$ sudo ss -tnlp | grep 8080
```

4. Copy the admin password to your clipboard:

```
$ argocd admin initial-password -n argocd | head -1 | pbcopy
```



Alternative command:

```
$ k -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d | pbcopy
```

5. Test the login / logout using the CLI:

```
$ argocd login --insecure localhost:8080
Username: admin
Password:
```



```
'admin:login' logged in successfully  
Context 'localhost:8080' updated
```

```
$ argocd logout --insecure localhost:8080  
Logged out from 'localhost:8080'
```

6. At this point you may want to follow the lab [Getting started with Argo CD](#).

3. Download and "install" this tutorial

1. Download [the zip for this tutorial](#) and extract it using the following commands:

```
$ zip=argo-cd-tutorial.zip; curl -sLO https://paulojeronimo.com/tmp/$zip  
$ d=${zip%.zip}; unzip -d $d $zip && cd $d
```

2. "Install" the tutorial with the these instructions:

```
$ export TUT_HOME=$PWD  
$ echo "!!" >> ~/.bashrc
```



So, technically, there is no installation for this tutorial. Only the `TUT_HOME` variable is added to your `~/.bashrc` file.

4. Spring Boot initial project creation

1. Install **spring**:

```
$ sdk install springboot
```

2. Generate the Spring Boot application we'll evolve in this tutorial:

```
$ spring init \  
  --type=gradle-project \  
  --name=echo-api \  
  --groupId=com.example \  
  --artifactId=echo-api \  
  --package-name=com.example.echo \  
  --description="Simple REST API with Java 25" \  
  --java-version=25 \  
  --boot-version=3.5.6 \  
  --build=gradle \  
  --dependencies=web \  
echo-api
```

3. Check the generated directory structure:

```
$ tree echo-api/  
echo-api/  
|-- build.gradle  
|-- gradle  
|   |-- wrapper  
|       |-- gradle-wrapper.jar  
|       |-- gradle-wrapper.properties  
|-- gradlew  
|-- gradlew.bat  
|-- HELP.md  
|-- settings.gradle  
|-- src  
|   |-- main  
|       |-- java  
|           |-- com  
|               |-- example  
|                   |-- echo  
|                       |-- EchoApiApplication.java  
|       |-- resources  
|           |-- application.properties  
|           |-- static  
|           |-- templates  
|-- test  
    |-- java
```

```
    '-- com
      '-- example
        '-- echo
          '-- EchoApiApplicationTests.java
```

17 directories, 10 files

4. Make your first initial Git commit:

```
$ cd echo-api
$ git init
$ git add .
$ git commit -m 'Initial commit'
```

5. Application development with Java 25 features

5.1. Gradle Wrapper update

1. Check the current **gradlew** version (by default, it is not compatible with **Java 25**):

```
$ ./gradlew --version

-----
Gradle 8.14.3
-----

Build time:    2025-07-04 13:15:44 UTC
Revision:     e5ee1df3d88b8ca3a8074787a94f373e3090e1db

Kotlin:       2.0.21
Groovy:       3.0.24
Ant:          Apache Ant(TM) version 1.10.15 compiled on August 25 2024
Launcher JVM: 25 (Eclipse Adoptium 25+36-LTS)
Daemon JVM:   /home/pj/.sdkman/candidates/java/25-tem (no JDK specified, using
current Java home)
OS:           Linux 6.8.0-85-generic amd64
```

2. Update the **gradle** wrapper:

```
$ ./gradlew wrapper --gradle-version 9.1.0 --distribution-type bin
```

3. Check the **gradlew** version again:

```
$ ./gradlew --version
Downloading https://services.gradle.org/distributions/gradle-9.1.0-bin.zip
.....10%.....20%.....30%.....40%.....50%....
.....60%.....70%.....80%.....90%.....100%

Welcome to Gradle 9.1.0!

Here are the highlights of this release:
- Full Java 25 support
- Native task graph visualization
- Enhanced console output

For more details see https://docs.gradle.org/9.1.0/release-notes.html
```

```
-----  
Gradle 9.1.0  
-----
```

```
Build time:    2025-09-18 13:05:56 UTC  
Revision:     e45a8dbf2470c2e2474ccc25be9f49331406a07e  
  
Kotlin:       2.2.0  
Groovy:       4.0.28  
Ant:          Apache Ant(TM) version 1.10.15 compiled on August 25 2024  
Launcher JVM: 25 (Eclipse Adoptium 25+36-LTS)  
Daemon JVM:   /home/pj/.sdkman/candidates/java/25-tem (no JDK specified, using  
current Java home)  
OS:           Linux 6.8.0-85-generic amd64
```

4. Make a new Git commit:

```
$ git status  
$ git add gradle  
$ git commit -m 'Update gradle version'
```

5. Read the [Gradle references](#) to learn more.

5.2. Creating the EchoController class

1. Create the `EchoController` class:

```
$ d=src/main/java/com/example/echo/controller; \  
(cd $TUT_HOME/code; rsync -R $d/EchoController.java $OLDPWD)
```

2. Here is the content of this class:

```
package com.example.echo.controller;  
  
import org.springframework.web.bind.annotation.*;  
import java.net.InetAddress;  
import java.time.ZonedDateTime;  
import java.util.Map;  
  
@RestController  
@RequestMapping("/api")  
public class EchoController {  
  
    @GetMapping("/echo")  
    public Map<String, Object> echo(@RequestParam String msg) throws Exception {  
        var serverIp = InetAddress.getLocalHost().getHostAddress();  
        var timestamp = ZonedDateTime.now().toString();  
    }  
}
```

```

        return Map.of(
            "message", msg,
            "serverIp", serverIp,
            "timestamp", timestamp
        );
    }
}

```

3. Run the application:

```

$ log=$(basename $PWD).log
$ ./gradlew bootRun &> $log &
$ tail -f $log

```

4. Press `Ctrl-C` to kill tail (this will not kill the Spring Boot application).

5. Test the results produced by this API:

```

$ curl -s 'localhost:8080/api/echo?msg=test' | jq .
{
  "timestamp": "2025-10-12T17:29:47.905602102+01:00[Europe/Lisbon]",
  "serverIp": "127.0.1.1",
  "message": "test"
}

```

6. Stop the application:

```

$ kill %1
$ jobs
[1]+  Exit 143                  ./gradlew bootRun &> $log

```

7. Make your second Git commit:

```

$ rm HELP.md
$ cat > .gitignore <<EOF
*.log
.classpath
.gradle
.project
.settings
build/
EOF
$ git add .
$ git commit -m 'Added EchoController'

```

5.3. Updating the EchoController class adding a Java 25 feature

TODO

6. Labs

6.1. Test a Kubernetes cluster with a load balancer

1. Go to `$TUT_HOME` and create a directory named `lab1`:

```
$ cd $TUT_HOME; mkdir lab1 && cd $_
```

2. Create a file configuration for a cluster with 3 nodes:

```
$ cat > kind-3nodes.yaml <<EOF
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: k3
networking:
  apiServerAddress: "127.0.0.1"
  apiServerPort: 6443
nodes:
  - role: control-plane
  - role: worker
  - role: worker
EOF
```

3. Create the cluster:

```
$ kind create cluster --config kind-3nodes.yaml
```

4. Check the cluster. You should see one control-plane and two workers:

```
$ k get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-
IP	OS-IMAGE		KERNEL-VERSION		CONTAINER-RUNTIME	
k3-control-plane	Ready	control-plane	59s	v1.34.0	172.18.0.2	<none>
Debian GNU/Linux 12 (bookworm)		6.14.0-33-generic		containerd://2.1.3		
k3-worker	Ready	<none>	48s	v1.34.0	172.18.0.4	<none>
Debian GNU/Linux 12 (bookworm)		6.14.0-33-generic		containerd://2.1.3		
k3-worker2	Ready	<none>	48s	v1.34.0	172.18.0.3	<none>
Debian GNU/Linux 12 (bookworm)		6.14.0-33-generic		containerd://2.1.3		

5. Create a deployment file for Nginx:

```
$ cat > nginx-deploy.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: nginx-demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-demo
  template:
    metadata:
      labels:
        app: nginx-demo
    spec:
      initContainers:
        - name: init-index
          image: busybox:1.36
          command: ["sh", "-c"]
          args: ["echo Pod: $(hostname) > /usr/share/nginx/html/index.html"]
          volumeMounts:
            - name: web
              mountPath: /usr/share/nginx/html
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          volumeMounts:
            - name: web
              mountPath: /usr/share/nginx/html
      volumes:
        - name: web
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx-demo
  type: NodePort
  ports:
    - name: http
      port: 80
      targetPort: 80
      nodePort: 30080
EOF

```

6. Deploy the nginx cluster:

```
$ k apply -f nginx-deploy.yaml
```

```
deployment.apps/nginx-demo created
service/nginx-svc created
```

```
$ k rollout status deploy/nginx-demo
deployment "nginx-demo" successfully rolled out
```

```
$ k get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx-demo-cc547f8fc-84n84	1/1	Running	0	95s	10.244.2.2	k3-worker2
nginx-demo-cc547f8fc-jnrtq	1/1	Running	0	95s	10.244.1.3	k3-worker
nginx-demo-cc547f8fc-kwbrl	1/1	Running	0	95s	10.244.1.2	k3-worker

```
$ k get svc nginx-svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-svc	NodePort	10.96.173.180	<none>	80:30080/TCP	3m56s

7. Test the load balancing. There is two ways to do that.

a. First way: from inside the cluster ("simplest" form):

```
$ k run testbox --rm -it --image=busybox:1.36 --restart=Never -- \
  sh -c 'for i in $(seq 1 10); do wget -qO- http://nginx-svc; sleep 0.5; done'
```

All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.

If you don't see a command prompt, try pressing enter.

Pod: nginx-demo-cc547f8fc-jnrtq
Pod: nginx-demo-cc547f8fc-kwbrl
Pod: nginx-demo-cc547f8fc-jnrtq
Pod: nginx-demo-cc547f8fc-kwbrl
Pod: nginx-demo-cc547f8fc-jnrtq
Pod: nginx-demo-cc547f8fc-84n84
Pod: nginx-demo-cc547f8fc-jnrtq
pod "testbox" deleted from default namespace

b. Second way: from host (using **NodePort** (30080)):

```
$ workers=$(docker ps --format '{{.Names}}' | grep k3-worker))

$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
${workers[*]}
172.18.0.4
172.18.0.3

$ worker_ips=$(!!)
```

```

worker_ips=$(docker inspect -f '{{range
.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' ${workers[*]})

$ for ip in ${worker_ips[*]}
do
  for i in {1..5}
  do
    echo -n "ip: $ip - "
    curl -s http://$ip:30080
    sleep 0.5
  done
done
ip: 172.18.0.4 - Pod: nginx-demo-cc547f8fc-84n84
ip: 172.18.0.4 - Pod: nginx-demo-cc547f8fc-84n84
ip: 172.18.0.4 - Pod: nginx-demo-cc547f8fc-kwbrl
ip: 172.18.0.4 - Pod: nginx-demo-cc547f8fc-84n84
ip: 172.18.0.4 - Pod: nginx-demo-cc547f8fc-jnrtq
ip: 172.18.0.3 - Pod: nginx-demo-cc547f8fc-jnrtq
ip: 172.18.0.3 - Pod: nginx-demo-cc547f8fc-84n84
ip: 172.18.0.3 - Pod: nginx-demo-cc547f8fc-84n84
ip: 172.18.0.3 - Pod: nginx-demo-cc547f8fc-kwbrl
ip: 172.18.0.3 - Pod: nginx-demo-cc547f8fc-84n84

```

8. Test the cluster using a Ingress Controller:

```

$ kind delete cluster --name k3
Deleting cluster "k3" ...

$ cat > kind-3nodes.ingress.yaml <<EOF
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: k3
nodes:
  - role: control-plane
    extraPortMappings:
      - containerPort: 80
        hostPort: 80
        protocol: TCP
      - containerPort: 443
        hostPort: 443
        protocol: TCP
  - role: worker
  - role: worker
EOF

$ kind create cluster --config kind-3nodes.ingress.yaml

$ k apply -f https://raw.githubusercontent.com/kubernetes/ingress-
nginx/main/deploy/static/provider/kind/deploy.yaml

```

```

$ k wait --namespace ingress-nginx \
  --for=condition=Available deploy/ingress-nginx-controller \
  --timeout=120s
deployment.apps/ingress-nginx-controller condition met

$ cat > nginx-deploy.ingress.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-demo
  template:
    metadata:
      labels:
        app: nginx-demo
    spec:
      initContainers:
        - name: init-index
          image: busybox:1.36
          command: ["sh", "-c"]
          args: ["echo Pod: $(hostname) > /usr/share/nginx/html/index.html"]
          volumeMounts:
            - name: web
              mountPath: /usr/share/nginx/html
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          volumeMounts:
            - name: web
              mountPath: /usr/share/nginx/html
      volumes:
        - name: web
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx-demo
  ports:
    - name: http
      port: 80
      targetPort: 80

```

EOF

```
$ k apply -f nginx-deploy.ingress.yaml
deployment.apps/nginx-demo created
service/nginx-svc created
```

```
$ k get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE	READINESS GATES					
nginx-demo-cc547f8fc-2gwx7	1/1	Running	0	3m32s	10.244.2.4	k3-
worker <none>	<none>					
nginx-demo-cc547f8fc-ccf9s	1/1	Running	0	3m32s	10.244.1.3	k3-
worker2 <none>	<none>					
nginx-demo-cc547f8fc-fg6rn	1/1	Running	0	3m32s	10.244.1.4	k3-
worker2 <none>	<none>					

```
$ k get svc nginx-svc
```

```
$ cat > nginx-ingress.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-demo
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: localhost
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx-svc
                port:
                  number: 80
```

EOF

```
$ k apply -f nginx-ingress.yaml
ingress.networking.k8s.io/nginx-demo created
```

```
$ for i in {1..10}; do curl -s http://localhost/; done
```

```
Pod: nginx-demo-cc547f8fc-6ldpf
Pod: nginx-demo-cc547f8fc-flfp7
Pod: nginx-demo-cc547f8fc-6ldpf
Pod: nginx-demo-cc547f8fc-flfp7
Pod: nginx-demo-cc547f8fc-m76pf
Pod: nginx-demo-cc547f8fc-6ldpf
Pod: nginx-demo-cc547f8fc-flfp7
```

```
Pod: nginx-demo-cc547f8fc-m76pf
Pod: nginx-demo-cc547f8fc-m76pf
Pod: nginx-demo-cc547f8fc-6ldpf
```

9. Some other commands you can test (to make questions!):

```
$ diff -uNr kind-3nodes{,.ingress}.yaml
$ diff -uNr nginx-deploy{,.ingress}.yaml
$ k get pods -l app=nginx-demo
$ k get endpoints nginx-svc -o wide
$ k logs deploy/nginx-demo
$ k describe svc nginx-svc
$ k describe deploy nginx-demo
$ $TUT_HOME/code/setup-kind-ingress-demo.sh
```

6.2. Getting started with Argo CD

```
$ argocd login --insecure localhost:8080
Username: admin
Password:
'admin:login' logged in successfully
Context 'localhost:8080' updated
```

Ref: https://argo-cd.readthedocs.io/en/stable/getting_started/#5-register-a-cluster-to-deploy-apps-to-optional

```
$ k config get-contexts -o name
kind-k3

$ argocd cluster add $(!!)
WARNING: This will create a service account `argocd-manager` on the cluster referenced
by context `kind-k3` with full cluster level privileges. Do you want to continue
[y/N]? y
{"level":"info","msg":"ServiceAccount `argocd-manager` created in namespace `kube-
system`","time":"2025-10-16T10:43:06+01:00"}
{"level":"info","msg":"ClusterRole `argocd-manager-role` created","time":"2025-10-
16T10:43:06+01:00"}
{"level":"info","msg":"ClusterRoleBinding `argocd-manager-role-binding`
created","time":"2025-10-16T10:43:06+01:00"}
{"level":"info","msg":"Created bearer token secret `argocd-manager-long-lived-token`
for ServiceAccount `argocd-manager`","time":"2025-10-16T10:43:06+01:00"}
{"level":"fatal","msg":"rpc error: code = Unknown desc = error getting server version:
failed to get server version: Get `https://127.0.0.1:6443/version?timeout=32s`: dial
tcp 127.0.0.1:6443: connect: connection refused","time":"2025-10-16T10:43:07+01:00"}
```

Ref: https://argo-cd.readthedocs.io/en/stable/getting_started/#creating-apps-via-cli

```
$ k config set-context --current --namespace=argocd
Context "kind-k3" modified.
```

```
$ argocd app create guestbook --repo https://github.com/argoproj/argocd-example-
apps.git --path guestbook --dest-server https://kubernetes.default.svc --dest
-namespace default
application 'guestbook' created
```

```
$ argocd app list
```

NAME	CLUSTER	NAMESPACE	PROJECT	STATUS
HEALTH	SYNCPOLICY	CONDITIONS	REPO	
PATH	TARGET			
argocd/guestbook	https://kubernetes.default.svc	default	default	OutOfSync
Missing	Manual	<none>	https://github.com/argoproj/argocd-example-apps.git	
guestbook				

```
$ argocd app get guestbook
```

Name: argocd/guestbook
Project: default
Server: https://kubernetes.default.svc
Namespace: default
URL: https://localhost:8080/applications/guestbook
Source:
- Repo: https://github.com/argoproj/argocd-example-apps.git
 Target:
 Path: guestbook
SyncWindow: Sync Allowed
Sync Policy: Manual
Sync Status: OutOfSync from (0d521c6)
Health Status: Missing

GROUP	KIND	NAMESPACE	NAME	STATUS	HEALTH	HOOK	MESSAGE
	Service	default	guestbook-ui	OutOfSync	Missing		
apps	Deployment	default	guestbook-ui	OutOfSync	Missing		

Ref: https://argo-cd.readthedocs.io/en/stable/getting_started/#syncing-via-cli

```
$ argocd app sync guestbook
```

TIMESTAMP	GROUP	KIND	NAMESPACE	NAME
STATUS	HEALTH	HOOK	MESSAGE	
2025-10-16T11:10:49+01:00		Service	default	guestbook-ui
OutOfSync	Missing			
2025-10-16T11:10:49+01:00	apps	Deployment	default	guestbook-ui
OutOfSync	Missing			

```
Name: argocd/guestbook
```


Project: default
Server: https://kubernetes.default.svc
Namespace: default
URL: https://localhost:8080/applications/guestbook
Source:
- Repo: https://github.com/argoproj/argocd-example-apps.git
Target:
Path: guestbook
SyncWindow: Sync Allowed
Sync Policy: Manual
Sync Status: Synced to (0d521c6)
Health Status: Progressing

Operation: Sync
Sync Revision: 0d521c6e049889134f3122eb32d7ed342f43ca0d
Phase: Succeeded
Start: 2025-10-16 11:10:49 +0100 WEST
Finished: 2025-10-16 11:10:49 +0100 WEST
Duration: 0s
Message: successfully synced (all tasks run)

GROUP	KIND	NAMESPACE	NAME	STATUS	HEALTH	HOOK	MESSAGE
	Service	default	guestbook-ui	Synced			service/guestbook-ui created
apps	Deployment	default	guestbook-ui	Synced			deployment.apps/guestbook-ui created

7. References

Ubuntu QEMU/KVM

1. <https://documentation.ubuntu.com/server/how-to/virtualisation/qemu/>

Java 25

1. <https://openjdk.org/jeps/465>

IntelliJ Idea

1. <https://blog.jetbrains.com/idea/2025/09/java-25-lts-and-intellij-idea/>

Gradle

1. https://docs.gradle.org/current/userguide/compatibility.html#java_runtime

Spring

1. <https://spring.io/blog/2025/09/16/this-week-in-spring-september-16th-2025>
2. <https://spring.io/guides/gs/spring-boot-docker>

Docker

1. <https://docs.docker.com/engine/install/ubuntu/#install-using-the-convenience-script>
2. <https://docs.docker.com/engine/install/linux-postinstall/>
3. <https://github.com/docker/docker-install>

Kubectl

1. <https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/>

Kind

1. <https://kind.sigs.k8s.io/docs/user/quick-start/#installation>

Argo CD

1. https://argo-cd.readthedocs.io/en/stable/getting_started/
2. https://argo-cd.readthedocs.io/en/stable/cli_installation/
3. https://argo-cd.readthedocs.io/en/stable/try_argo_cd_locally/

SDKMAN

1. <https://sdkman.io/>